LAB 3 (ANALISIS Y DISEÑO)

Cristian Miguel Jimenez
Mecatronica
(ITLA)
La CALETA, Boca Chica
20240712@itla.edu.do

***Abstract*— En esta práctica de laboratorio utilizamos las diferentes compuertas lógicas del campo de la electrónica digital en conjunto con la tabla de verdad y el mapa de Karnaugh para lograr hacer un circuito lógico combinacional que resuelva una problemática específica de un parqueo de un complejo de apartamentos.**

Abstract*— *In this laboratory practice we use the different logic gates from the field of digital electronics in conjunction with the truth table and the Karnaugh map to make a combinational logic circuit that solves a specific problem in a parking lot in an apartment complex.

Keywords*— *K-maps, Tabla de verdad, compuertas lógicas.

I. INTRODUCCION

En este proyecto se diseñó un sistema sencillo para un parqueadero de tres puestos. La idea es que cada espacio tiene un sensor que detecta si está ocupado (1) o libre (0), y según eso se enciende un solo LED indicando cuál es el primer espacio libre disponible, siguiendo el orden del primero al último. Si todos los puestos están ocupados, no se enciende ningún LED. Se usó lógica combinacional para crear una tabla de verdad y luego se implementó el circuito usando compuertas en el simulador Gowin. Con esto, se busca aplicar conceptos básicos de diseño digital de una manera práctica y clara.

II. MARCO TEORICO

Circuito Logico Combinacional

Un circuito lógico combinacional es aquel en el que el estado actual de la combinación de las entradas lógicas decide la salida. El término lógica combinacional significa la combinación de dos o más puertas lógicas para formar una función requerida donde la salida en un momento dado depende solo de la entrada.

Tabla De Verdad

Una tabla de verdad en electrónica digital es una representación sistemática y exhaustiva de las combinaciones posibles de valores de entrada y las correspondientes salidas en un circuito lógico.

En un circuito digital, las señales de entrada pueden ser 0 (falso) o 1 (verdadero), y la salida también puede ser 0 o 1, dependiendo de las condiciones lógicas establecidas por el diseño del circuito.

K-Maps

Los Mapas de Karnaugh, también llamados «K-maps», son

una representación gráfica utilizada para simplificar expresiones booleanas en el diseño de circuitos lógicos digitales. Funcionan organizando las diferentes combinaciones de variables en un formato que facilita la identificación de patrones, los cuales, a su vez, permiten reducir complejas ecuaciones booleanas a su forma más simple.

Materiales utilizados:

Proto

Jumper

FPGA

Led's

Resistencias

Simuladores:

Gowin

Multisim

DESARROLLO

Un individuo tiene 4 espacios de estacionamiento afuera de su apartamento. Este individuo quiere saber cuándo dos espacios adyacentes están libres, ya que no quiere que nadie se estacione junto a su carro. (Normalmente, esta persona se estaciona al final de un gran lote de estacionamiento para evitar rayaduras o golpes).

Este individuo decide instalar interruptores de presión en los espacios de estacionamiento y tener un indicador lógico en su apartamento. Desea que un indicador lógico se active si dos espacios adyacentes están libres, otro indicador lógico si tres espacios adyacentes están libres, un tercero si todos los espacios están libres y un cuarto si solo un espacio está libre (en caso de que decida arriesgarse).

ABCD	L1	L2	L3	L4
0000	0	0	0	1
0001	0	0	1	0
0010	0	1	0	0
0011	0	1	0	0
0100	0	1	0	0
0101	0	0	0	0
0110	0	0	0	0
0111	1	0	0	0
1000	0	0	1	0
1001	0	1	0	0
1010	0	0	0	0
1011	1	0	0	0
1100	0	1	0	0
1101	1	0	0	0
1110	1	0	0	0
1111	0	0	0	0

Ahora creamos nuestra tabla de verdad tomando en cuenta el problema del individuo y que nuestros (0) representan los espacios libres y nuestros (1) los espacios ocupados.

Fig 1. Tabla De Verdad

Ahora realizamos nuestros K-maps con (Min-términos) y las formulas.

K-maps de la salida P0 con Min-términos:

P0				
AB/CD	00	01	11	10
00			1	1
01	1			
11	1			
10		1		

A'B'C + BC'D' + AB'C'D

K-maps salida P1 con Min-términos:

P1				
AB/CD	00	01	11	10
00		1		
01				
11				
10				

A'B'C'D

K-maps salida P2 con Min-términos:

P2				
AB/CD	00	01	11	10
00				
01				
11		1		1
10	1		1	

AB'C'D' + AB'CD + ABC'D + ABCD'

K-maps salida P3 con Min-términos:

P3				
AB/CD	00	01	11	10
00	1			
01				
11				
10				

A'B'C'D'

Luego de tener nuestras formulas con (Min-términos) realizamos nuestro circuito combinacional en el simulador multisim:

Ahora realizamos nuestros K-maps con Max-términos.

K-maps de la salida P0 con Max-términos:

AB\CD	00	01	11	10
00	0	0	0	0
01	0	0	1	0
11	0	1	0	1
10	0	0	1	0

(A+C)(B+C)(B+D)(C+D)(A+B)(A+D)(A+B+C+D)

K-maps de la salid P1 con Max-términos:

AB\CD	00	01	11	10
00	0	0	1	1
01	1	0	0	0
11	1	0	0	0
10	0	1	0	0

(A+B+C)(B*+D*)(B*C*)(A*+C*)(A*+B+D)

K-maps de la salida P2 con Max-términos:

AB\CD	00	01	11	10
00	0	1	0	0
01	0	0	0	0
11	0	0	0	0
10	1	0	0	0

(AD)(B+C)(A+D')(C)

K-maps de la salida P3 con Max-términos:

AB\CD	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	0	0	0	0
10	0	0	0	0

(A)(B)(C)(D)

Luego que tenemos nuestros K-maps y las formulas realizamos nuestro circuito combinacional con Max-términos en el simulador de multisim:

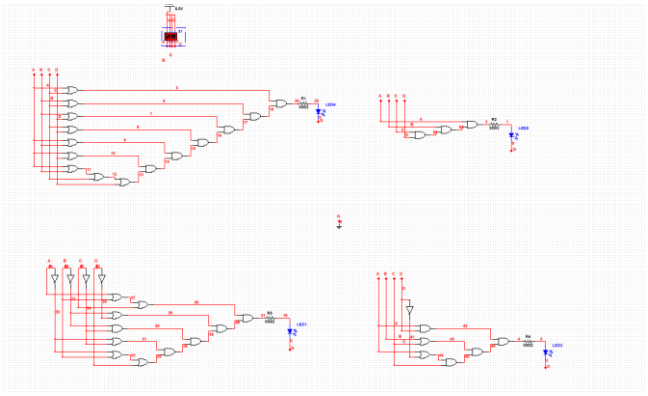


Fig 3. Circuito combinacion con Max-términos en multisim.

Si su matricula termina en un numero par debe implementar en el FPGA con Max-terminos de lo contrario (impar) con min-terminos.

Como mi matricula termina en 01, el 1 es impar, por lo tanto utilizo min-términos.

Aqui el codigo

```
module controlParqueo(
    input A,B, C, D,
    output LED1, LED2, LED3, LED4
    );

    // Señales negadas de las entradas
    wire notA, notB, notC, notD;

    // Bloque de compuertas NOT para invertir las entradas
    not_1 negaciones(
        .A1(A), .Y1(notA),
        .A2(B), .Y2(notB),
        .A3(C), .Y3(notC),
        .A4(D), .Y4(notD)
    );

    // Resultados de combinaciones OR
    // La letra "n" indica que la señal está negada (ejemplo: Bn = r
    wire orAB, orBnDn, orBnCn, orAnCn;

    // Primer grupo de compuertas OR
    or_1 opcsA1(
        .A1(A), .B1(B), .Y1(orAB),
        .A2(notB), .B2(notD), .Y2(orBnDn),
        .A3(notB), .B3(notC), .Y3(orBnCn),
        .A4(notA), .B4(notC), .Y4(orAnCn)
    );

    // Más combinaciones OR intermedias
    wire orAnB, orABC, orAnBD;

    // Segundo grupo de compuertas OR
    or_2 opcsB1(
```

Puede usar fpga, pero creando modulos de compuertas e instanciandolas.

Primero la AND

```
module and_1(
    input A1, B1,
    output Y1, // Salidas de la compuerta AND 1

    input A2, B2,
    output Y2, // Salidas de la compuerta AND 1

    input A3, B3,
    output Y3, // Salidas de la compuerta AND 1

    input A4, B4,
    output Y4  // Salidas de la compuerta AND 1
);
    // Implementación de cuatro compuertas AND independie
    and (Y1, A1, B1);
    and (Y2, A2, B2);
    and (Y3, A3, B3);
    and (Y4, A4, B4);
```

AND de nuestro P0

Luego tenemos nuestra instancacion de P1

```
module and_2(
    input A1, B1,
    output Y1, // Salida de la compuerta AND 1

    input A2, B2,
    output Y2, // Salida de la compuerta AND 2

    input A3, B3,
    output Y3  // Salida de la compuerta AND 3
);
    // Implementación de tres compuertas AND independientes
    and (Y1, A1, B1);
    and (Y2, A2, B2);
    and (Y3, A3, B3);
endmodule
```

AND P1

Instanciacion de NOT

```
module not_1(
    input A1,
    output Y1, // Salida de la compuerta NOT interna 1

    input A2,
    output Y2, // Salida de la compuerta NOT interna 2

    input A3,
    output Y3, // Salida de la compuerta NOT interna 3

    input A4,
    output Y4  // Salida de la compuerta NOT interna 4
);
    // Implementación de cuatro compuertas NOT independientes
    not (Y1, A1);
    not (Y2, A2);
    not (Y3, A3);
    not (Y4, A4);
endmodule
```

NOT

```
module not_1(
    input A1,
    output Y1, // Salida de la compuerta NOT interna 1

    input A2,
    output Y2, // Salida de la compuerta NOT interna 2

    input A3,
    output Y3, // Salida de la compuerta NOT interna 3

    input A4,
    output Y4  // Salida de la compuerta NOT interna 4
);
    // Implementación de cuatro compuertas NOT independientes
    not (Y1, A1);
    not (Y2, A2);
    not (Y3, A3);
    not (Y4, A4);
endmodule
```

Intanciacion de OR

OR en P0 Instanciacion de OR

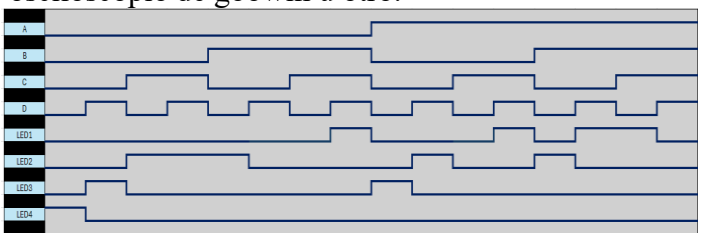
```
module or_2(
    input A1, B1,
    output Y1, // Salida de la compuerta OR interna 1

    input A2, B2,
    output Y2, // Salida de la compuerta OR interna 2

    input A3, B3,
    output Y3  // Salida de la compuerta OR interna 3
);
    // Implementación de tres compuertas OR independientes
    or (Y1, A1, B1);
    or (Y2, A2, B2);
    or (Y3, A3, B3);
endmodule
```

OR en P2

Diagrama de tiempo usando
osciloscopio de goowin u otro.



CONCLUSION

Esta practica me hice dormir mal
y tener dos ataques de nervios
pero aprendí el uso del fpga y
con fe todo se pudo lograr

Keep your text and graphic files separate until after the text has been formatted and styled. Do not use hard tabs, and limit use of hard returns to only one return at the end of a paragraph. Do not add any kind of pagination anywhere in the paper. Do not number text heads-the template will do that for you.

A. Abbreviations and Acronyms

Define abbreviations and acronyms the first time they are used in the text, even after they have been defined in the abstract. Abbreviations such as IEEE, SI, MKS, CGS, sc, dc, and rms do not have to be defined. Do not use abbreviations in the title or heads unless they are unavoidable.

B. Units

- Use either SI (MKS) or CGS as primary units. (SI units are encouraged.) English units may be used as secondary units (in parentheses). An exception would be the use of English units as identifiers in trade, such as “3.5-inch disk drive”.
- Avoid combining SI and CGS units, such as current in amperes and magnetic field in oersteds. This often leads to confusion because equations do not balance dimensionally. If you must use mixed units, clearly

may use the solidus (/), the exp function, or appropriate

We suggest that you use a text box to insert a graphic (which is ideally a 300 dpi TIFF or EPS file, with all fonts embedded) because, in an MSW document, this method is somewhat more stable than directly inserting a picture.

To have non-visible rules on your frame, use the MSWord “Format” pull-down menu, select Text Box > Colors and Lines to choose No Fill and No Line.